Available online at www.sciencedirect.com
SCIENCE DIRECT®
Computers & Operations Research 33 (2006) 1823–1837
computers & operations research
www.elsevier.com/locate/cor

Solving capacitated arc routing problems using a transformation to the CVRP

Humberto Longo[a], Marcus Poggi de Aragão[b,*], Eduardo Uchoa[c]

[a]*Instituto de Informática, Universidade Federal de Goiás, Campus Samambaia, Goiania, GO 74001-970, Brazil*
[b]*Departamento de Informática, Pontifícia Universidade Católica do Rio de Janeiro, R. Marquês de São Vicente 225, Rio de Janeiro, RJ 22453-900, Brazil*
[c]*Departamento de Engenharia de Produção, Universidade Federal Fluminense, Rua Passo da Pátria 156, Niterói, RJ 24210-240, Brazil*

Abstract

A well-known transformation by Pearn, Assad and Golden reduces a capacitated arc routing problem (CARP) into an equivalent capacitated vehicle routing problem (CVRP). However, that transformation is regarded as unpractical, since an original instance with r required edges is turned into a CVRP over a complete graph with $3r + 1$ vertices. We propose a similar transformation that reduces this graph to $2r + 1$ vertices, with the additional restriction that a previously known set of r pairwise disconnected edges must belong to every solution. Using a recent branch-and-cut-and-price algorithm for the CVRP, we observed that it yields an effective way of attacking the CARP, being significantly better than the exact methods created specifically for that problem. Computational experiments obtained improved lower bounds for almost all open instances from the literature. Several such instances could be solved to optimality.

Scope and purpose The scope of this paper is transforming arc routing problems into node routing problems. The paper shows that this approach can be effective and, in particular, that the original instances may generate node routing instances that behave as if the size is not increased. This result is obtained by slightly modifying the well-known transformation by Pearn, Assad and Golden from capacitated arc routing problem (CARP) to the capacitated vehicle routing problem (CVRP), that is regarded as unpractical. The paper provides a computational experience using a recent branch-and-cut-and-price algorithm for the CVRP. The results are significantly better

* Corresponding author. Tel.: +55 21 3114 1510; fax: +55 21 3114 1530.
E-mail addresses: longo@inf.ufg.br (H. Longo), poggi@inf.puc-rio.br (M. Poggi de Aragão), uchoa@producao.uff.br (E. Uchoa).

0305-0548/$ - see front matter © 2004 Elsevier Ltd. All rights reserved.
doi:10.1016/j.cor.2004.11.020

than the exact methods created specifically for that problem, improving lower bounds for almost all open instances from the literature. Several such instances could be solved to optimality.
© 2004 Elsevier Ltd. All rights reserved.

Keywords: Arc routing; Mixed-integer programming

1. Introduction

The capacitated arc routing problem (CARP) can be defined as follows. Suppose a connected undirected graph $G = (V, E)$, costs $c : E \to Z^+$, demands $w : E \to Z^+$, vehicle capacity Q and a distinguished depot vertex labeled 0. Define $R = \{e \in E \mid w(e) > 0\}$ as the set of required edges. Let F be a set of closed walks that start and end at the depot, where edges in a walk can be either *serviced* or *deadheaded*. Set F is a feasible CARP solution if:

- each required edge is serviced by exactly one walk in F;
- the sum of demands of the serviced edges in each walk in F does not exceed the vehicle capacity.

We want to find a solution minimizing the sum of the costs of the walks. It can be noted that $\sum_{e \in R} c(e)$ is a trivial lower bound on the cost of an optimal solution, the remaining costs in a solution are the costs of the deadheaded edges. In the remaining of this article, let r denote the number of edges in R.

This problem was first presented by Golden and Wong in 1981 [1] and has been used to model many situations, including street garbage collection, postal delivery, routing of electric meter readers, etc. [2].

The CARP is strongly NP-hard. Several heuristics have been proposed for it. Among them we can cite Golden et al. [3], Chapleau et al. [4], Ulusoy [5], Pearn [6,7] and Hertz et al. [8]. Many other heuristics are described in [9,10,2].

On the other hand, as far as we know, the only exact algorithms for it are the branch-and-bound algorithms by Hirabayashi et al. [11], Kiuchi et al. [12], Welz [13], and Belenguer and Benavent [14]. Even using the fast machines available today, those algorithms can only solve small instances, with less than 30 required edges.

Algorithms yielding lower bounds for the CARP are presented by Golden and Wong [1], Assad et al. [15], Pearn [16], Benavent et al. [17], Amberg and Voß [18] and Wøhlk [19]. The current best bounds are those obtained by a cutting plane algorithm by Belenguer and Benavent [20]. Those bounds are much better than previous ones and matched the best heuristic solutions on 47 out of 87 instances tested from the literature. The largest instance thus solved had 97 required edges. The algorithm uses cuts over z variables, where $z(e)$ represents the number of times edge e is deadheaded. However, those cuts are not enough to give a *formulation* for the CARP, they are only a *relaxation* since they allow some integer solutions that do not correspond to feasible solutions. This complicates the use of those bounds in an exact algorithm because *no one knows how to determine in polynomial time whether an integer solution z is feasible or not*. In other words, that bound can at most prove that a given heuristic CARP solution is indeed optimal, and only on instances without duality gap.

The approach proposed in this work, reducing the CARP to a CVRP that is given as input to a branch-and-cut-and-price algorithm, not only gives lower bounds even better than those by Belenguer and Benavent [20], it is already an exact algorithm that can solve many open instances from the literature.

2. Pearn, Assad and Golden's transformation

Pearn et al. [21] proposed a transformation of the CARP to the CVRP that replaces each edge of R by three vertices. An edge (i, j) in R is associated to vertices s_{ij} and s_{ji}, referred as side vertices, and to m_{ij}, the middle vertex. A CVRP instance is defined on the complete undirected graph $H = (N, A)$, where

$$N = \bigcup_{(i,j)\in R} \{s_{ij}, s_{ji}, m_{ji}\} \cup \{0\}.$$

Vertex 0 is defined as the depot. The edge costs $d : A \to Z^+$ and the demands $q : N \to Z^+$ are defined as follows:

$$d(s_{ij}, s_{kl}) = \begin{cases} \frac{1}{4}(c_{ij} + c_{kl}) + dist(i, k) & \text{if } (i, j) \neq (k, l), \\ 0 & \text{if } (i, j) = (k, l), \end{cases}$$

$$d(0, s_{ij}) = \tfrac{1}{4}\, c_{ij} + dist(0, i),$$

$$d(m_{ij}, v) = \begin{cases} \frac{1}{4}\, c_{ij} & \text{if } v = s_{ij} \text{ or } s_{ji}, \\ \infty & \text{otherwise,} \end{cases}$$

where $dist(i, j)$ is the value of the shortest path from vertex i to vertex j in G, calculated using the costs c. The new demands are

$$q(s_{ij}) = q(m_{ij}) = q(s_{ji}) = \tfrac{1}{3}\, w(i, j).$$

The resulting instance has $3r + 1$ vertices. Since the current best CVRP codes [22–27] can only cope consistently with instances up to 100 vertices, the practical use of that transformation is limited.

3. New transformation

We observed that, in the above transformation, the construction around the middle vertex (m_{ij}) has only the purpose of obliging a CVRP route to pass through the three vertices corresponding to edge (i, j) in sequence (either $s_{ij} \to m_{ij} \to s_{ji}$ or $s_{ji} \to m_{ij} \to s_{ij}$). Our idea to avoid the middle vertices is simply to fix r edges to 1 in the transformed CVRP instance. The new transformation is described as follows.

An edge (i, j) in R is now associated only to vertices s_{ij} and s_{ji}. The resulting CVRP instance is defined on the complete undirected graph $H = (N, A)$, where

$$N = \bigcup_{(i,j)\in R} \{s_{ij}, s_{ji}\} \cup \{0\}.$$

Vertex 0 is defined as the depot. The edge costs $d : A \to Z^+$ and the demands $q : N \to Z^+$ are defined as follows:

$$d(s_{ij}, s_{kl}) = \begin{cases} 0 & \text{if } (i, j) = (k, l), \\ c(i, j) & \text{if } (i, j) = (l, k), \\ dist(i, k) & \text{if } (i, j) \neq (k, l), (i, j) \neq (l, k), \end{cases}$$

$$d(0, s_{ij}) = dist(0, i),$$

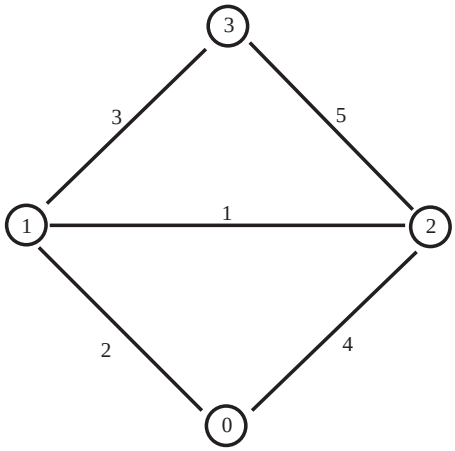


Fig. 1. Original CARP graph.

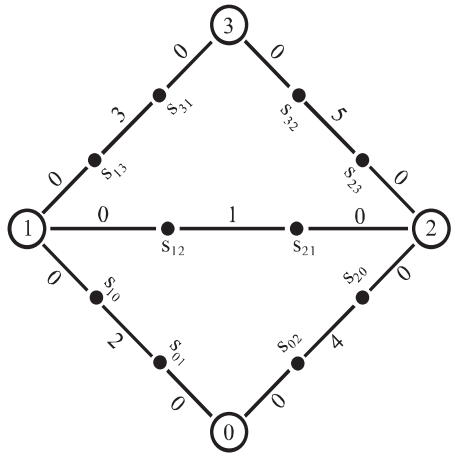


Fig. 2. CARP graph with vertices s_{ij}.

where $dist(i, j)$ have the same meaning as in the previous section. The new demands are

$$q(s_{ij}) = q(s_{ji}) = \tfrac{1}{2}\, w(i, j).$$

In fact, as in the previous transformation [21], we can split the demand in an arbitrary way, as long as $q(s_{ij}) + q(s_{ji}) = w(i, j)$.

Finally, we require that all edges $\{(s_{ij}, s_{ji}) \mid (i, j) \in R\}$ belong to the solution, i.e. we only accept CVRP solutions where s_{ij} and s_{ji} are visited in sequence, either $s_{ij} \to s_{ji}$ or $s_{ji} \to s_{ij}$.

We give as example of the transformation an instance with 4 vertices and 5 edges (all required), where $V = \{0, 1, 2, 3\}$ and edge costs are $c(0, 1) = 2$, $c(0, 2) = 4$, $c(1, 2) = 1$, $c(1, 3) = 3$ and $c(2, 3) = 5$, see Fig. 1. The transformation introduces vertices s_{ij} and s_{ji} for all required edges $(i, j) \in R$, as shown in Fig. 2. The final transformed instance, a constrained CVRP instance with 11 vertices, is defined over a

Table 1
Internode distances of the resulting CVRP graph in Fig. 3

	0	s_{01}	s_{10}	s_{02}	s_{20}	s_{12}	s_{21}	s_{13}	s_{31}	s_{23}	s_{32}
0	—	0	2	0	3	2	3	2	5	3	5
s_{01}		—	2	0	3	2	3	2	5	3	5
s_{10}			—	2	1	0	1	0	3	1	3
s_{02}				—	4	2	3	2	5	3	5
s_{20}					—	1	0	1	4	0	4
s_{12}						—	1	0	3	1	3
s_{21}							—	1	4	0	4
s_{13}								—	3	1	3
s_{31}									—	4	0
s_{23}										—	5
s_{32}											—

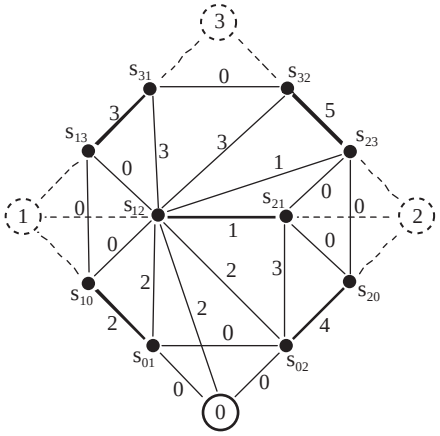


Fig. 3. Resulting CVRP graph. A complete graph (distances in Table 1). Vertices 1, 2, and 3 are not in the CVRP graph. Several edges are omitted: $(0, s_{31})$, $(0, s_{32})$, $(0, s_{13})$, (s_{02}, s_{32}), (s_{01}, s_{31}), etc.

complete graph with the distances between nodes presented in Table 1. Fig. 3 depicts the resulting CVRP graph, several of the total of 55 edges are omitted for a better visualization.

An important point is that having to impose the r edges (s_{ij}, s_{ji}) to belong to the solution is not an additional burden to most CVRP algorithms. Quite to the contrary, they ease a lot the solution of a transformed instance. Although the transformation results in an instance where $|N|=2r+1$, the mandatory use of this set of edges can make an algorithm to perform almost as if on an instance with $|N| = r + 1$. We address this point in the next section.

4. Formulation

The goal of this section is to present how a CVRP formulation can be slightly specialized to instances coming from the CARP, in order to take advantage of having r pairwise disconnected edges that must belong to every solution.

The explicit master formulation for the CVRP, as presented in [27], combines a column generation formulation with the classical formulation on edge variables. Let $H = (N, A)$, d, q and Q define a CVRP instance having vertex 0 as the depot and the remaining vertices in N as clients.

This formulation follows:

$$\min \quad \sum_{e=(u,v)\in A} d(e)x_e \tag{0}$$

$$\text{s.t.} \quad \sum_{e\in\delta(\{u\})} x_e = 2 \quad \forall u \in N\backslash\{0\}, \tag{1}$$

$$\sum_{e\in\delta(\{0\})} x_e \geqslant 2K^*, \tag{2}$$

$$\text{EM-CVRP}: \quad \sum_{e\in\delta(S)} x_e \geqslant 2k(S) \quad \forall S \subseteq N\backslash\{0\}, \tag{3}$$

$$x_e \leqslant 1 \quad \forall e \in A\backslash\delta(\{0\}), \tag{4}$$

$$\sum_{l=1}^{p} q_l^e \lambda_l - x_e = 0 \quad \forall e \in A, \tag{5}$$

$$x_e \in \{0, 1, 2\}, \quad \forall e \in A, \tag{6}$$

$$\lambda_l \geqslant 0 \quad \forall l \in \{1, \ldots, p\}.$$

- Variable x_e represents the number of times that edge e is traversed by a vehicle. This variable can assume value 2 if e is adjacent to the depot, corresponding to a route with a single client.
- The λ_l variables would ideally be associated with valid routes. This would imply having a strongly NP-hard column generation problem. We associate the λ_l variables to the set of all *q-routes* satisfying the vehicle capacity constraint. A *q-route* is a walk that starts at the depot, traverses a sequence of clients with total demand at most Q, and returns to the depot. Clients may appear more than once in a *q-route* and its demand considered for each time. Each variable λ_l is therefore associated to one of the p possible q-routes. Let q_l^e be the number of times edge e appears in the lth q-route. Remark that the column generation problem for *q-routes* can be solved by a pseudo-polynomial algorithm. This contribution is due to Christofides et al. [28] that used *q-routes* in a Lagrangean relaxation-based algorithm for the CVRP.
- Degree constraints (1) states that each client vertex is served by exactly one vehicle. Constraint (2) requires that at least K^* vehicles leave and return to the depot. This number, representing the minimum number of vehicles to service all clients, is calculated by solving a Bin-Packing Problem. The *rounded capacity* constraints stated in (3) use $k(S) = \lceil \sum_{u\in S} q(u)/Q \rceil$ as a lower bound on the minimum number of vehicles necessary to service the clients in set $S \subset N$. Constraints (5) oblige x to be a linear combination of q-routes. The integrality constraints complete the formulation.

Now we proceed with the simplifications that come from the fact that $H = (N, A)$, d, q and Q comes from a CARP instance, as defined in the previous section, and that all edges $\{(s_{ij}, s_{ji}) \mid (i, j) \in R\}$ are fixed to 1.

- Since all vehicles appearing in a CARP solution must service at least one edge in R and that each edge is represented as two vertices in H, we can restrict all x variables to be less or equal to 1.
- We can restrict the q-routes to those that visit vertices s_{ij} and s_{ji} in sequence, either $s_{ij} \rightarrow s_{ji}$ or $s_{ji} \rightarrow s_{ij}$. This eliminates many λ variables. Pricing q-routes over a complete graph with n vertices takes $O(n^2 Q)$ time. The above restriction does not change that complexity.
- Half of constraints (1) can be eliminated. The redefinition of the q-routes and constraints (5) guarantee that if the degree constraint corresponding to s_{ij} is satisfied, the degree constraint corresponding to s_{ji} will also be satisfied. This observation has a crucial impact on the performance of the resulting branch-and-cut-and-price, because the size of LPs and the convergence of the column generation part of the algorithm depends a lot on the number of such constraints. In other words, in those aspects the algorithm behaves as if it were solving an instance with r clients and not $2r$.
- It is well-known (see for instance [24,29]) that when separating capacity cuts, edges (u, v) having value 1 in the fractional solution should be contracted. This happens because if a capacity cut over a set S having $(u, v) \in \delta(S)$ is violated, a capacity cut over set $S \cup \{u, v\}$ is also violated. This means that the corresponding separation algorithm always work on a graph with at most $r + 1$ vertices.

This specialized formulation, called EM-CARP, is now presented:

$$\min \quad \sum_{e=(u,v)\in A} d(e)x_e \tag{0}$$

$$\text{s.t.} \quad \sum_{e\in\delta(\{s_{ij}\})} x_e = 2 \quad \forall (i, j) \in R, \tag{1'}$$

$$\sum_{e\in\delta(\{0\})} x_e \geqslant 2K^*, \tag{2}$$

$$\text{EM-CARP}: \quad \sum_{e\in\delta(S)} x_e \geqslant 2k(S) \quad \forall S \subseteq N\backslash\{0\}, \tag{3}$$

$$x_e \leqslant 1 \quad \forall e \in A, \tag{4'}$$

$$\sum_{l=1}^{p'} q_l^e \lambda_l - x_e = 0 \quad \forall e \in A, \tag{5}$$

$$x_e \in \{0, 1\}, \quad \forall e \in A, \tag{6}$$

$$\lambda_l \geqslant 0 \quad \forall l \in \{1, \ldots, p'\}.$$

A more compact formulation, the one actually used in the algorithm, is obtained if every occurrence of x_e in (0), (1′), (2), (3) is replaced by its equivalent given by (5). Relaxing the integrality constraints,

a linear program, referred DWM-CARP, is obtained. The solution of that LP gives a valid lower bound.

$$\min \quad \sum_{l=1}^{p'} \left(\sum_{e \in A} d(e) q_l^e \right) \lambda_l \tag{7}$$

$$\text{s.t.} \quad \sum_{l=1}^{p'} \left(\sum_{e \in \delta(\{s_{ij}\})} q_l^e \right) \lambda_l = 2 \quad \forall (i, j) \in R, \tag{8}$$

$$\text{DWM-CARP}: \quad \sum_{l=1}^{p'} \left(\sum_{e \in \delta(\{0\})} q_l^e \right) \lambda_l \geqslant 2K^*, \tag{9}$$

$$\sum_{l=1}^{p'} \left(\sum_{e \in \delta(S)} q_l^e \right) \lambda_l \geqslant 2k(S) \quad \forall S \subseteq N \backslash \{0\}, \tag{10}$$

$$\sum_{l=1}^{p'} q_l^e \lambda_l \leqslant 1 \quad \forall e \in A, \tag{11}$$

$$\lambda_l \geqslant 0 \quad \forall l \in \{1, \ldots, p'\}.$$

Not only rounded capacity inequalities can appear in the DWM-CARP, a generic cut $\sum_{e \in A} a_e x_e \geqslant b$ can be included as $\sum_{l=1}^{p'} \left(\sum_{e \in A} a_e q_l^e \right) \lambda_l \geqslant b$. In this way, other valid inequalities, the ones described in [24] for instance, were added to this formulation and tested.

This same approach can be applied to incorporate the inequalities proposed in Belenguer and Benavent [20]. These inequalities are defined on variables z representing the number of times an arc is deadheaded. We need only to translate each of these variables into the x variables of the original formulation. This can be easily done, since the edges (s_{ij}, s_{kl}) stand for the shortest path from vertex i to vertex k in the original graph, which is known and therefore allows listing all edges that are deadheaded. For example, edge (s_{21}, s_{02}), in Fig. 3, is associated to the (shortest)path (2, 1), (1, 0), in Fig. 1.

The translation of an inequality on the z variables to one on the x variables can then be achieved by the substitution of every variable z_e, e an edge of the original graph, by the sum of all variables x_{uv} such that the shortest path from u to v contains edge e. Let $Z_e(x)$ denote this expression. Consider, for example, the *odd edge cutset constraints* in [20]. Take the odd edge cutset defined by vertex 2 in Fig. 1. The *odd edge cutset constraint* is then $z_{02} + z_{12} + z_{23} \geqslant 1$. The equivalent constraint on x variables, $Z_{02}(x) + Z_{12}(x) + Z_{23}(x) \geqslant 1$ writes

$$x_{s_{21}s_{32}} + x_{s_{20}s_{32}} + x_{s_{21}s_{31}} + x_{s_{20}s_{31}} + x_{s_{23}s_{31}} + x_{s_{21}s_{10}} + x_{s_{20}s_{10}} + x_{s_{23}s_{10}} + x_{s_{21}s_{01}} + x_{s_{20}s_{01}} + x_{s_{23}s_{01}} + x_{s_{20}s_{12}} + x_{s_{23}s_{12}} + x_{s_{21}s_{13}} + x_{s_{20}s_{13}} + x_{s_{23}s_{13}} + x_{s_{21}s_{02}} + x_{s_{23}s_{02}} \geqslant 1,$$

where the left-hand side stands for the expression $Z_{12}(x)$ that corresponds to variable z_{12}, while variables z_{02} and z_{23} have an empty expressions $Z_{02}(x)$ and $Z_{23}(x)$, respectively.

The translation above gives an efficient, although not polynomial time, way to test whether the solution given by the formulation on the z variables of Belenguer and Benavent [20] is a feasible solution to the CARP. Let z^* represent an optimal integer solution of the formulation proposed in [20]. All that has to be done is to check whether the system on the x variables given by $Z_e(x) = z_e^* \ \forall e \in A$, together with

(1), (2), (3), (4′) and $x_{s_{ij},s_{ji}} = 1\ \forall (i, j) \in R$ has a feasible 0-1 solution. If the answer is affirmative, then the objective function value given by z^* is optimal. Otherwise it will correspond to a lower bound. In the former case, an optimal solution for the CARP associated to z^* can be constructed from any $\overline{x}$ satisfying the system just defined. This construction is straightforward.

5. The branch-and-cut-and-price algorithm

The algorithm is a branch-and-bound algorithm on the DWM-CARP formulation, which has an exponential number of variables and constraints as well. We next describe the main modules of this algorithm.

5.1. Column generation

The reduced cost of a column (λ variable) is the sum of the reduced costs of the edges in the corresponding q-route. Let μ, ν, π, and ω be the dual variables associated with constraints (8), (9), (10), and (11), respectively. The reduced cost $\bar{d}_e$ of an edge e is given by

$$\bar{d}_e = \begin{cases} d_e - \mu_i - \mu_j - \omega_e - \sum\limits_{S|\delta(S)\ni e} \pi_S, & e = \{i, j\} \in E\backslash\delta(\{0\}), \\ d_e - \nu - \mu_j - \sum\limits_{S|\delta(S)\ni e} \pi_S, & e = \{0, j\} \in \delta(\{0\}). \end{cases}$$

An additional generic cut $\sum_{e\in E} a_e x_e \geqslant b$ in DWM-CARP, associated with dual variable α, contributes with the value $-a_e \cdot \alpha$ to the computation of $\bar{d}_e$.

Since the degree constraints (8) are only present for vertices s_{ij}, and omitted for the corresponding s_{ji} vertices, the search for q-routes must be restricted to the ones that visits in sequence s_{ij} and s_{ji}, or s_{ji} and s_{ij}, every time one such vertex is reached. We call these restricted q-routes, qr-routes.

The pricing subproblem becomes finding qr-routes (columns) of minimum reduced cost, i.e. qr-routes for which the sum of the edges is minimum. This can be done in pseudopolynomial time [28].

Remember that a q-route admits visiting a same vertex more than once, consequently a qr-route will allow traversing a same required edge more than once while considering its demand for each time as well. Therefore, using qr-routes instead of valid routes leads to a worst lower bound. We attenuate this drawback by further restricting the qr-routes to not having cycles of small size, say, two or three.

In our implementation a cycle of size two corresponds to visiting vertex s_{ij}, follow to a vertex s_{kl}, meaning that the route went to s_{lk} and then to s_{kl}, and then back again to s_{ij} through s_{ji}. In other words, eliminating such two cycles only forbids traversing a required edge in the same sense it has been traversed just before traversing the last required edge.

Using the technique presented in Irnich and Villeneuve [30], we implemented the elimination of cycles of size three, i.e. a requested edge can only be visited in the same sense after the visit of three other requested edges (including the same requested edge in the opposite sense). The computational experience is done on the algorithm eliminating these 3 cycles.

5.2. Cut generation

The algorithm starts only with constraints (8) and (9) in the LP formulation. The remaining constraints are treated as cuts that are added as needed. Besides the bound cuts (11) and the well-known rounded

capacity cuts (10), we also tried the framed capacity, strengthened comb, multistar, partial multistar, generalized multistar and hypotour cuts, all presented in [24], using the CVRPSEP package [31] to separate them. As usual, cuts are defined on variables associated to the edges of the underlying graph. These separation procedures are then applied to the optimal solution of the LP relaxation used. This solution is denoted x_e^* for all edges e. Since we are using the DWM-CARP formulation, these variables are not readily available. The optimal linear relaxation solution, given in terms of its variables λ, is represented by λ_j^* for all qr-routes j. Therefore we have to convert variables λ_j^*, $j = 1, \ldots, p'$ into x_e^*, $e \in A$, to apply the CVRPSEP separation procedures.

A preliminary computational experience has shown that the only really effective cuts, besides (10) and (11), where the framed capacity and strengthened comb cuts. As a result only these cuts are present in our computational experience.

5.3. Branching rule

The algorithm starts by adding columns to the formulation until there are none with negative reduced costs, then it looks for all violated cuts. This is repeated until both column generation and separation fail. If the node cannot be fathomed at this point, we branch.

The branching rule is an adaptation of the rule used in [24]. We choose as branching candidates sets S such that $2 < x^*(\delta(S)) < 4$ and branch by imposing the disjunction $(x(\delta(S)) = 2) \vee (x(\delta(S)) \geqslant 4)$. Priority is given to sets with smaller values of $|x^*(\delta(S)) - 2.7|/d(S)$. The value of 2.7 was chosen because adding the constraint $x(\delta(S)) = 2$ tends to have a greater impact on the LP relaxation than $x(\delta(S)) \geqslant 4$, leading to an imbalance in the branch-and-bound tree. Values closer to 2 than 4 increase the impact of imposing $x(\delta(S)) \geqslant 4$.

5.4. Node selection and primal bounds

We used as initial primal bound the best previously known solution plus one. The node selection strategy chosen was *depth first search*, because it requires less memory, and is equivalent to any other node selection strategy if the optimal solution value is known in advance.

6. Computational results

Our CARP code is basically an adaptation of the implementation of a robust branch-and-cut-and-price algorithm for the CVRP described in [27]. Apart from a new pricing algorithm to restrict the q-routes to those visiting vertices s_{ij} and s_{ji} in sequence, we only changed the linear programming part to be like indicated in DWM-CARP. Tests were conducted on a Pentium IV, 2.4 GHz, with 1 GB of RAM.

In the computational experiment we applied our algorithm to the instances of the datasets *kshs*, *gdb*, *bccm* and *egl*, available at http://www.uv.es/∼belengue/carp.html. These datasets were originally used in Kiuchi et al. (*kshs*) [12], DeArmon [32] and Golden et al. (*gdb*) [3], Benavent et al. (*bccm*) [17], and Li [33] and Li and Eglese (*egl*) [34], respectively. Except for the *bccm* instances, that are named $1.A, 1.B, \ldots, 10D$, the remaining three sets have their names starting with the names of the sets.

Sets *kshs*, *gdb* and *bccm* were randomly generated following different construction patterns, varying the underlying graph, the vehicle capacity or the capacities itself. In these three sets of instances all edges

Table 2
Lower bound comparison for the egl instances

Instance	$\|V\|$	$\|E\|$	r	K^*	UB	Prev. LB	Our LB	Root time (s)
egl-e1-a	77	98	51	5	3548	3515	**3548**	144.203
egl-e1-b	77	98	51	7	4498	4436	4468	52.594
egl-e1-c	77	98	51	10	5595	5453	5542	46.063
egl-e2-a	77	98	72	7	5018	4994	5011	521.219
egl-e2-b	77	98	72	10	6340	6249	6280	198.531
egl-e2-c	77	98	72	14	8415	8114	8234	66.875
egl-e3-a	77	98	87	8	5898	5869	**5898**	924.859
egl-e3-b	77	98	87	12	7822	7646	7697	375.609
egl-e3-c	77	98	87	17	10,433	10,019	10,163	142.125
egl-e4-a	77	98	98	9	6461	6372	6395	1171.580
egl-e4-b	77	98	98	14	9021	8809	8884	418.984
egl-e4-c	77	98	98	19	11,779	11,276	11,427	202.688
egl-s1-a	140	190	75	7	5018	4992	5014	750.391
egl-s1-b	140	190	75	10	6435	6201	6379	204.500
egl-s1-c	140	190	75	14	8518	8310	8480	66.984
egl-s2-a	140	190	147	14	9995	9780	9824	3260.250
egl-s2-b	140	190	147	20	13,174	12,886	12,968	896.706
egl-s2-c	140	190	147	27	16,795	16,221	16,353	408.878
egl-s3-a	140	190	159	15	10,296	10,025	10,143	1680.371
egl-s3-b	140	190	159	22	14,053	13,554	13,616	1639.530
egl-s3-c	140	190	159	29	17,297	16,969	17,100	635.292
egl-s4-a	140	190	190	19	12,442	12,027	12,143	14,318.100
egl-s4-b	140	190	190	27	16,531	15,933	16,093	2761.130
egl-s4-c	140	190	190	35	20,832	20,179	20,375	1119.980
Average gap %						2.40	1.45	
Maximum gap %						4.27	3.11	

are required, i.e. R equals E. The fourth set, *egl*, was constructed using as underlying graph regions of the road network of the county of Lancashire (UK). Costs and demands are proportional to the length of the edges, except for non-required edges that have zero demand.

In the first two tables we compare the lower bound given by the root node of our branch-and-cut-and-price algorithm (the solution of DWM-CARP) with the best known lower bounds of all instances from sets *egl* (Table 2) and *bccm* (Table 3). The columns of both tables list the name of the instance and its characteristics, the number of vertices $|V|$, the number of required edges r, the minimum number of vehicles K^* to cover the demands, the best known upper bound *UB*, the previous best lower bound (all of them obtained in [20]), the lower bound given by our transformation *Our LB*, and the corresponding CPU time *RootTime* in seconds. Lower bounds in bold indicate a matching with the best upper bound.

The results show that our lower bounds are equal or better than the previous best on almost all instances. On only two instances, $2.B$ and $5.B$, our bound was one unit smaller. It can be observed that our bounds are strictly better on all instances where $K^* > 5$, indicating that its quality is less sensitive to the use of many vehicles. The new bounds are also strictly better on all *egl* instances.

Table 3
Lower bound comparison for the bccm instances

Instance	$\|V\|$	r	K^*	UB	Prev. LB	Our LB	Root time (s)
1.A	24	39	2	247	**247**	**247**	98.278
1.B	24	39	3	247	**247**	**247**	54.561
1.C	24	39	8	319	309	312	771.882
2.A	24	34	2	298	**298**	**298**	79.404
2.B	24	34	3	330	**330**	329	169.010
2.C	24	34	8	528	526	**528**	0.969
3.A	24	35	2	105	**105**	**105**	127.590
3.B	24	35	3	111	**111**	**111**	134.339
3.C	24	35	7	162	161	161	3.156
4.A	41	69	3	522	**522**	**522**	2475.280
4.B	41	69	4	534	**534**	**534**	1178.370
4.C	41	69	5	550	**550**	**550**	824.599
4.D	41	69	9	652	644	648	76.576
5.A	34	65	3	566	**566**	**566**	629.417
5.B	34	65	4	589	**589**	588	388.144
5.C	34	65	5	617	612	613	274.772
5.D	34	65	9	724	714	716	62.779
6.A	31	50	3	330	**330**	**330**	158.667
6.B	31	50	4	340	338	337	169.254
6.C	31	50	10	424	418	420	119.371
7.A	40	66	3	382	**382**	**382**	319.349
7.B	40	66	4	386	**386**	**386**	163.754
7.C	40	66	9	437	436	436	607.637
8.A	30	63	3	522	**522**	**522**	359.426
8.B	30	63	4	531	**531**	**531**	168.885
8.C	30	63	9	663	653	654	376.863
9.A	50	92	3	450	**450**	**450**	17,722.600
9.B	50	92	4	453	**453**	**453**	4520.030
9.C	50	92	5	459	**459**	**459**	1460.890
9.D	50	92	10	518	509	512	305.158
10.A	50	97	3	637	**637**	**637**	13,336.600
10.B	50	97	4	645	**645**	**645**	13,719.400
10.C	50	97	5	655	**655**	**655**	5078.570
10.D	50	97	10	739	732	734	473.632
Average gap %					0.41	0.32	
Maximum gap %					3.13	2.19	

The last table (Table 4) presents the instances where we could run the complete branch-and-cut-and-price algorithm, solving them to optimality. The columns of this table have the same first four columns of the previous two tables. They are followed by columns with the optimal value of the instance *OPT*, the CPU time of our algorithm at the root node *ROOT Time*, the number of nodes in the branch-and-bound tree *Tree Nodes*, and the CPU time spent in the column generation algorithm *CG Time*, the cut generation procedures *Cut Time* and total CPU time *Total Time*. The optimal values that are in bold indicate that

Table 4
Results of the BCP algorithm for the kshs, gdb, bccm and egl instances

Instance	$\|V\|$	r	K^*	Opt	Root time (s)	Tree nodes	CG time (s)	Cut time (s)	Total time (s)
kshs1	8	15	4	14,661	0.719	2	0.656	0.172	0.828
kshs2	10	15	4	9863	0.375	2	0.391	0.047	0.453
kshs3	6	15	4	9320	0.688	4	0.766	0.219	1.016
kshs4	8	15	4	**11,498**	0.828	3	0.531	0.391	0.953
kshs5	8	15	3	10,957	0.703	1	0.656	0.047	0.703
kshs6	9	15	3	10,197	0.734	3	1.391	0.078	1.484
gdb1	12	22	5	316	1.859	10	1.359	1.672	3.125
gdb2	12	26	6	339	0.531	5	1.125	0.141	1.359
gdb3	12	22	5	275	0.406	5	0.531	0.234	0.844
gdb4	11	19	4	287	0.281	8	0.625	0.125	0.813
gdb5	13	26	6	377	0.891	2	0.422	0.578	1.016
gdb6	12	22	5	298	0.250	1	0.203	0.047	0.250
gdb7	12	22	5	325	0.625	10	1.203	0.422	1.750
gdb8	27	46	10	**348**	2.750	28	30.922	0.859	32.469
gdb9	27	51	10	303	12.016	7	22.141	8.313	30.969
gdb10	12	25	4	275	1.063	10	5.500	0.656	6.344
gdb11	22	45	5	395	19.438	29	1351.130	9.688	1364.630
gdb12	13	23	7	**458**	0.703	27	4.422	0.406	5.219
gdb13	10	28	6	**536**	1.125	145	88.063	1.484	93.031
gdb14	7	21	5	100	0.156	1	0.156	0	0.156
gdb15	7	21	4	58	0.781	9	3.094	0.531	3.750
gdb16	8	28	5	127	2.313	19	24.406	1.563	26.422
gdb17	8	28	5	91	0.953	23	33.328	0.703	34.828
gdb18	9	36	5	164	5.578	16	53.641	2.172	56.391
gdb19	8	11	3	55	0.063	1	0.047	0	0.063
gdb20	11	22	4	121	1.375	7	3.719	0.656	4.500
gdb21	11	33	6	156	3.969	13	17.781	2.734	20.859
gdb22	11	44	8	200	6.406	29	105.109	5.156	112.078
gdb23	11	55	10	233	9.547	30	136.438	6.625	145.453
1C	24	39	8	**319**	774.906	6119	7802.220	830.156	8916.78
2B	24	34	3	330	169.484	14	581.656	86.938	671.250
2C	24	34	8	**528**	0.938	1	0.891	0.047	0.953
3C	24	35	7	**162**	3.141	363	314.719	4.438	328.891
7C	31	66	9	**437**	39.781	3	54.609	9.500	131.969
egl-e1-a	77	51	5	**3548**	143.844	1	128.391	15.1719	143.875
egl-e3-a	77	87	8	**5898**	923.656	1	831.188	91.7969	923.766

optimality was proved for the first time. This was the case for 10 instances, the last one open from set *kshs*, the last three from set *gdb*, four from *bccm* and two from set *egl*. On one instance (gdb13) the optimal solution found improved upon the previous best heuristic solution (Table 4).

We also remark that our lower bound has matched the best known upper bound for instance 1.A, therefore proving its optimality. However, our algorithm itself was not able to find such optimal solution in reasonable time. It gets stuck by exploring nodes beyond depth 20 in the branch-and-bound tree.

Overall, the CPU times where reasonably small, which gives hope to closing some more of the remaining open instances on sets *bccm* and *egl* instances in the near future. Moreover, we believe that the lower bounds can be further improved with the addition of cuts that take into account the specific structure of the CARP, for example, the ones proposed in Belenguer and Benavent [20].

Acknowledgements

We would like to thank two anonymous referees for their helpful comments. HL was partially supported by CAPES/PICDT grant, MPA was partially supported by CNPq grant 302086/2003-0 and EU was partially supported by CNPq grant 304533/2002-5.

References

[1] Golden BL, Wong RT. Capacitated arc routing problems. Networks 1981;11:305–15.
[2] Dror M. Arc routing: theory, solutions and applications. Dordrecht: Kluwer Academic Publishers; 2000.
[3] Baker EK, DeArmon JS, Golden BL. Computational experiments with algorithms for a class of routing problems. Computers and Operations Research 1983;10(1):47–59.
[4] Chapleau L, Ferland JA, Lapalme G, Rousseau J-M. A parallel insert method for the capacitated arc routing problem. Operations Research Letters 1984;3(2):95–9.
[5] Ulusoy G. The Zeet size and mix problem for capacitated arc routing. European Journal of Operational Research 1985;22(3):329–37.
[6] Pearn W-L. Approximate solutions for the capacitated arc routing problem. Computers and Operations Research 1989;16(6):589–600.
[7] Pearn W-L. Argument-insert algorithms for the capacitated arc routing problem. Computers and Operations Research 1991;18(2):189–98.
[8] Hertz A, Laporte G, Mittaz M. A tabu search heuristic for the capacitated arc routing problem. Operations Research 2000;48(1):129–35.
[9] Assad AA , Golden BL. Arc routing methods and applications. In: Ball MG, Magnanti TL, Monma CL, Nemhauser GL, editors. Network routing, handbooks in operations research and management science, vol. 8. Amsterdam: Elsevier; 1995. p. 375–483 [chapter 5].
[10] Eiselt HA, Gendreau M, Laporte G. Arc routing problems II: the rural postman problem. Operations Research 1995;43(3):339–414.
[11] Hirabayashi R, Nishida N, Saruwatari Y. Node dublication lower bounds for the capacitated arc routing problems. Journal of the Operations Research Society of Japan 1992;35(2):119–33.
[12] Kiuchi M, Shinano Y, Hirabayashi R, Saruwatari Y. An exact algorithm for the capacitated arc routing problem using parallel branch and bound method. In: Abstracts of the 1995 spring national conference of the Operational Research Society of Japan. 1995. p. 28–9 [in Japanese].
[13] Welz SA. Optimal solutions for the capacitated arc routing problem using integer programming. PhD Thesis, Department of QT and OM, University of Cincinnati; 1994.
[14] Belenguer JM, Benavent E. The capacitated arc routing problem: valid inequalities and facets. Computational Optimization and Applications 1998;10(2):165–87.
[15] Assad AA, Golden BL, Pearn W. The capacitated Chinese postman problem: lower bounds and solvable cases. American Journal of Mathematics and Management Science 1987;7(1,2):63–88.
[16] Pearn W-L. New lower bounds for the capacitated arc routing problem. Networks 1988;18:181–91.
[17] Benavent E, Campos V, Corberan A, Mota E. The capacitated arc routing problem: lower bounds. Networks 1992;22: 669–90.
[18] Amberg A, Voß S. A hierarquical relaxations lower bound for the capacitated arc routing problem. In: Proceedings of the 35th annual Hawaii international conference on system sciences; 2002.

[19] Wϕhlk S. New lower bound for the capacitated arc routing problem. Preprint PP-2003-15, University of Southern Denmark, IMADA - Department of Mathematics and Computer Science, 2003.
[20] Belenguer JM, Benavent E. A cutting plane algorithm for the capacitated arc routing problem. Computers and Operations Research 2003; 705–28.
[21] Pearn WL, Assad A, Golden BL. Transforming arc routing into node routing problems. Computers and Operations Research 1987;14(4):285–8.
[22] Augerat P, Belenguer J, Benavent E, Corberán A, Naddef D, Rinaldi G. Computational results with a branch and cut code for the capacitated vehicle routing problem. Technical report 949-M, Université Joseph Fourier, Grenoble, France; 1995.
[23] Blasum U, Hochstättler W. Application of the branch and cut method to the vehicle routing problem. Technical report ZPR2000-386, Zentrum fur Angewandte Informatik Köln; 2000.
[24] Lysgaard J, Letchford A, Eglese R. A new branch-and-cut algorithm for the capacitated vehicle routing problem. Mathematical Programming 2004;100(2):423–45.
[25] Wenger KM. Generic cut generation methods for routing problems. PhD Thesis, Institute of Computer Science, University of Heidelberg; 2003.
[26] Ralphs T, Kopman L, Pulleyblank W, Trotter Jr L. On the capacitated vehicle routing problem. Mathematical Programming 2003;94:343–59.
[27] Fukasawa R, Lysgaard J, Poggi de Aragão M, Reis M, Uchoa E, Werneck RF. Robust branch-and-cut-and-price for the capacitated vehicle routing problem. In: Proceedings of the X IPCO, Lecture Notes in Computer Science, vol. 3064, New York; June 2004. pp. 1–15.
[28] Christofides N, Mingozzi A, Toth P. Exact algorithms for the vehicle routing problem, based on spanning tree and shortest path relaxations. Mathematical Programming 1981;20:255–82.
[29] Naddef D, Rinaldi G. Branch-and-cut algorithms for the capacitated VRP. In: Toth P, Vigo D., editors. The vehicle routing problem. Philadelphia: SIAM; 2002. p. 53–84 [chapter 3].
[30] Irnich S, Villeneuve D. The shortest path problem with resource constraints and k-cycle elimination for $k \geqslant 3$. INFORMS Journal on Computing 2004, to appear.
[31] Lysgaard J. CVRPSEP: a package of separation routines for the capacitated vehicle routing problem; 2003. Available at www.asb.dk/∼lys.
[32] DeArmon JS. A comparison of heuristics for the capacitated Chinese postman problem. Master's Thesis, University of Maryland, Colledge Park, MD; 1981.
[33] Li LYO. Vehicle routeing for winter gritting. PhD Thesis, Department of Management Science, Lancaster University; 1992.
[34] Li LYO, Eglese RW. An interactive algorithm for vehicle routeing for winter-gritting. Journal of the Operational Research Society 1996;47:217–28.